🎯 Reinforcement Learning (RL) 🎯

https://drive.google.com/file/d/1gWmitIlj7mam4ZmdYC5csS5JW7voF33-/view?pli=1

🤖 What is Reinforcement Learning?

Reinforcement Learning (RL) is a type of Machine Learning where an **agent learns by interacting with an environment.**

The agent:

- Performs actions,
- receives rewards or penalties,
- learns from experience,
- and improves its decisions over time.

The main idea is: **"Learn which actions give the highest rewards."**

RL solves **sequential decision-making problems.**

This means:

- actions are taken **one after another**
- each **action** affects the **future state**
- the **agent must decide the best sequence of actions to reach a goal**

🔹 Goal of RL

The objective is: **To learn to act optimally in the environment**

The agent should learn:

- what action to take,

- in which state,
- to get maximum reward.

🔹 Rewards and Feedback

After every action, the **agent receives feedback** called a **reward**.

- *Positive reward → good action*
- *Negative reward (penalty) → bad action*

The agent **learns from these rewards**

🔹 Utility of Agent

- **utility (performance)** of an RL agent is **defined by the rewards it collects.**
- **More rewards = better performance.**
- So the agent must: maximize expected rewards

🔹 Learning from Samples

- RL learns through **trial and error**.
- The agent:
    - observes outcomes,
    - stores experiences,
    - improves future decisions.

It does NOT need predefined correct answers.

🧩 Components of Reinforcement Learning

Component	Meaning

Agent	Learner/decision maker
Environment	World where agent operates
State (S)	Current situation
Action (A)	Move taken by agent
Reward (R)	Feedback received
Policy (π)	Strategy of choosing actions

🌍 Applications of Reinforcement Learning

🎮 Game Playing: Agents **learn winning strategies by playing repeatedly**.

🚗 Self-Driving Cars

RL helps cars: make driving decisions, avoid collisions, choose safe routes.

🤖 Robotics

Robots learn: Walking, object picking, Balancing, navigation.

📈 Stock Market & Trading

RL agents learn when to buy or sell, how to maximize profit.

🏥 Healthcare

- treatment planning, medicine dosage optimization, patient monitoring.

📦 Recommendation Systems

- Netflix etc use RL to improve recommendations.

🔄 Types of Reinforcement Learning

1️⃣Positive Reinforcement

- Good behavior is rewarded.
- **Example**
  - Robot reaches destination: Reward = +10
  - This increases chances of repeating the action.

2️⃣Negative Reinforcement

- **Undesirable condition** is **removed after correct action.**
- **Example**
  - Alarm stops when correct action is taken.
  - Encourages good behavior.

3️⃣Model-Based RL

- Agent has a **model of environment.**
- It can:
  - *predict outcomes,*
  - *plan future actions.*

4️⃣Model-Free RL

- Agent learns only from experience.
- No environment model is available.
- Example: **Q-Learning**

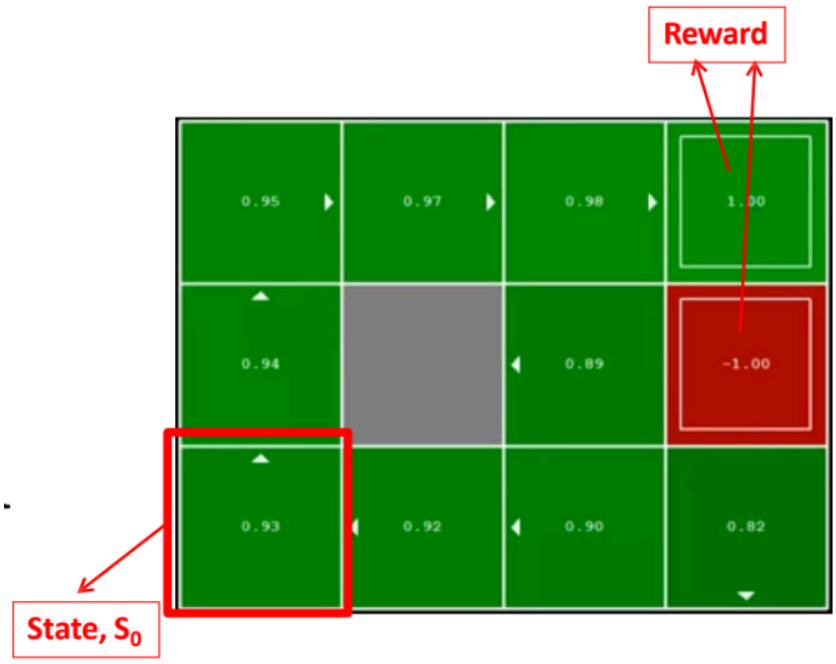
🧮 Markov Decision Process (MDP)

- RL problems are usually modeled using MDP
- MDP provides a **mathematical framework** for RL.

An MDP is defined by:

Component	Meaning
States (S)	Possible situations
Actions (A)	Possible actions
Transition Function T(s,a,s')	Probability of moving to next state
Reward Function R(s,a,s')	Reward received after transition

Sometimes **Start state (s0)** & **End/terminal state** are also included.



🧠 What Does "Markov" Mean?

- "Markov" means: **future depends only on the present state.**
- The **past does NOT matter if current state is known**.

📌 Markov Property

Instead of depending on all previous states: $P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots)$

the next state depends only on:

$$P(s_{t+1} \mid s_t, a_t)$$

Meaning: **Current state + current action completely determine future transition.**

📜 Policy in RL $\pi(s)$

- **Policy** is a **recommendation function**
- tells the agent **what action to take in a given state.**
- It is represented as: $\pi(s)$
- **Input** → current state
- **Output** → action to take
- The best policy is called**: optimal policy** π^*
- It is the policy that gives: maximum rewards, maximum utility, best long-term outcome.
- For MDPs, we want to achieve the optimal policy

🎯 Learning Optimality

- The goal of RL is: **Collect maximum rewards**
- The agent tries to maximize total reward over time.
- **'The optimality criteria is to maximize the rewards'**

💰 Total Reward (Undiscounted Utility)

If:

r_t = reward at time t

then total reward is:

$$r_0 + r_1 + r_2 \ldots = \sum_{t=0}^{\infty} r_t$$

(Undiscounted utility)

> All rewards are treated equally.

⏳ Discounted Utility

- In RL, **future rewards** are usually considered less important.
- So we multiply future rewards by a discount factor [**gamma**]:

$$0 \leq \gamma \leq 1$$

$$r_0 + \gamma r_1 + \gamma^2 r_2 \ldots = \sum_{t=0}^{\infty} \gamma^t r_t$$

Rewards farther in future become **highly discounted.**

γ Value	Meaning
γ = 0	Only immediate reward matters
γ close to 1	Future rewards also important

🌟 Value Functions

- **Value Function** tells us: **"How good or useful something is."**
- It **measures** the **expected reward** **an agent can receive in the future.**
- **utility(usefulness) of a sequence of rewards** is defined as a value function

Value functions help the agent:

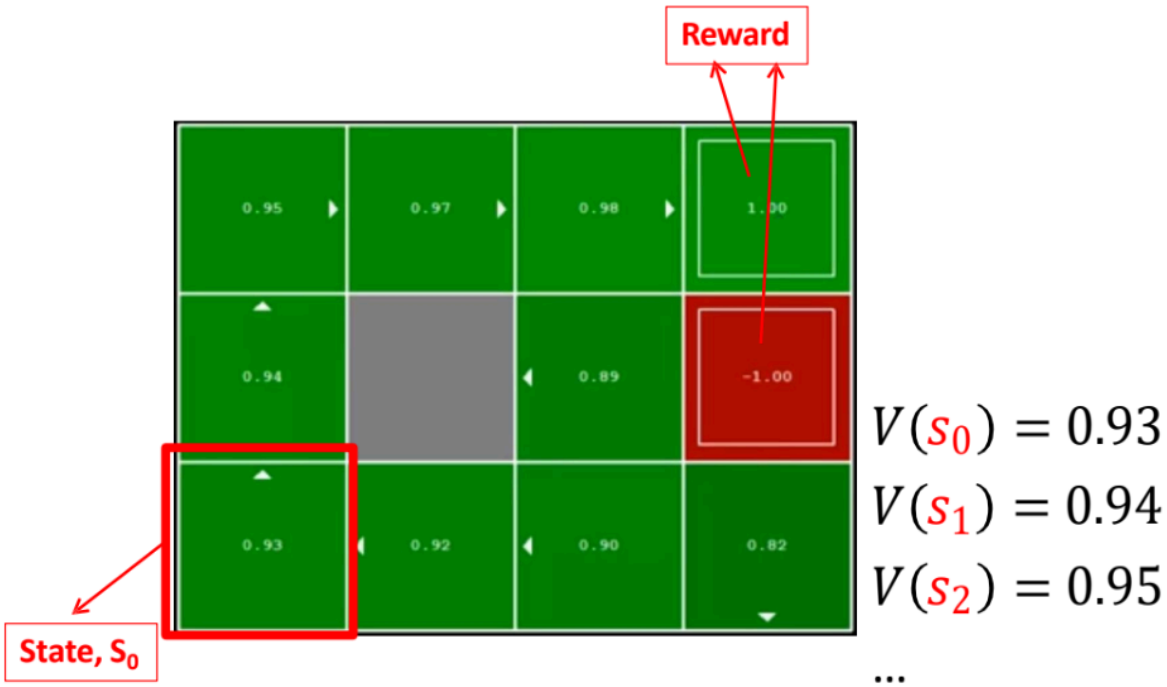
- evaluate states,
- evaluate actions,
- choose better decisions,
- maximize rewards.

🟢 Types of Value Functions

Type		Meaning
State Value Function	**V(s)**	Goodness of being in a state
Action Value Function	**Q(s,a) / V(s,a)**	Goodness of taking action (a) in state (s)

📍 State Value Function — V(s)

- A state value tells: **"How good is it to be in this state?"**
- It estimates the **future rewards expected from that state.**



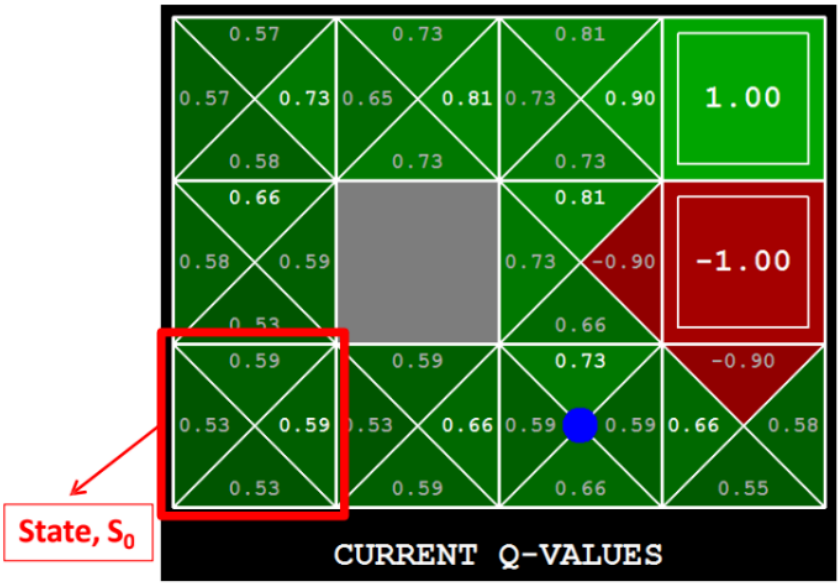
🔍 **Interpretation:** Higher value = better chance of receiving rewards.

State	Value	Meaning
(s_0)	0.93	Good state
(s_1)	0.94	Slightly better
(s_2)	0.95	Even better

Color	Meaning
Green	Positive/normal states
Red	Negative reward state
Gray	Obstacle/block
Bright Green	Goal state (+1 reward)

🔥 Q-Values (Action Value Function)

- A Q-value tells: **"How good is it to take action (a) from state (s)?"**
- Q-values evaluate specific actions.



$$Q\ (s_0, Up) = 0.59$$
$$Q\ (s_0, Right) = 0.59$$
$$Q\ (s_0, Down) = 0.53$$
$$Q\ (s_0, Left) = 0.53$$

Action	Q-Value	Meaning
Up	0.59	Good action
Right	0.59	Good action
Down	0.53	Less useful
Left	0.53	Less useful

The agent always prefers: **actions with higher Q-values.**

Because: **they lead to better long-term rewards.**

🤖 Reinforcement Learning Methods

🔹 Unknown Model in RL

- In Reinforcement Learning: **The environment model is usually unknown.**
- Specifically:
    - Transition function (T)
    - Reward function (R)

  are **NOT given.**

- The agent **must** learn through interaction.

📌 RL Methods Categories

RL methods mainly fall into:

Type	Approach
Model-Based RL	**Learn model first**
Model-Free RL	**Learn directly from experience**

1️⃣ Model-Based Reinforcement Learning

First learn:

- transition function (T)
- reward function (R)

by interacting with environment.

Then use them to: **compute optimal policy.** [thru Dynamic programming]

2️⃣Model-Free Reinforcement Learning

The agent directly learns:

- value functions,
- policies,

WITHOUT learning the environment model.

› Example: Q - learning

📌 **Learns From**

- trial and error,
- rewards,
- experience.

🤖 Q-Learning Algorithm

Q-Learning is a:

- **Model-free Reinforcement Learning algorithm**
- **Online learning algorithm**
- based on **trial and error**

It learns: "Which action is best in a particular state."

› Q-Learning **improves the estimates of Q-values** using:

1. *Immediate reward*
2. *Future discounted rewards*

The agent **continuously updates its knowledge after every action. [** Temporal difference learning**]**

> Q-Learning uses a concept called: **Temporal Difference Learning**

This means:

- learning happens **immediately after experience,**
- estimates are updated using **current reward + future estimate.**

Instead of waiting until the end: the agent learns step-by-step.

Agent:

1. observes current state
2. chooses action
3. performs action
4. receives reward
5. moves to next state
6. updates Q-value

This repeats continuously.

🧩 Action Selection Using Policy

The agent chooses action using:

$$\pi(s) = \arg\max Q(s, a)$$

- The **policy selects** the action with the **highest Q-value.**
- agent chooses: **the action expected to give maximum reward.**

🟢 Transition in Q-Learning

Suppose the agent:

- **starts in state (s)**
- **takes action (a)**
- **reaches next state (s')**

During this transition: it **receives reward (r)**

From this experience, the agent discovers:

- ✅ **Immediate reward** **r**
- ✅ **Next state's Q-values** **Q(s',a)**

This information is used to improve learning.

🎯 Sample Estimate

The sampled experience is:

$$\text{Sample} = r + \gamma \max_a Q(s', a)$$

Symbol	Meaning
r	**Immediate reward**
gamma	**Discount factor**
max Q(s',a)	**Best future reward**

new estimate should depend on: **current reward** PLUS **best possible future reward.**

🔥 Q-Learning Update Rule

$$Q(s,a) = Q(s,a) + \alpha[r + \gamma \max_a Q(s',a) - Q(s,a)]$$

› adds the **sample to** the **old estimate** of Q-value using a **learning rate α** as running average

› It updates old knowledge using:

- current experience,
- future expectation

$$0 \leq \alpha \leq 1$$

High α	**Learn quickly**
Low α	**Learn slowly**

🔄 Running Average Interpretation

$$Q(s,a) = \alpha(\text{sample}) + (1-\alpha)Q(s,a)$$

- part old knowledge,
- part new experience.

🧩 Q-Learning Algorithm Steps

Algorithm 1 Q-learning as proposed by Watkins and Dayan (1992)

Require: discount factor γ, learning rate α

1: Initialize Q(s,a) arbitrarily
2: **for all** episodes **do**
3: Initialize starting state s
4: **repeat**
5: Choose action a using Q and an exploration strategy
6: Execute a
7: Observe next state s' and recieve immediate reward r
8: Update, $Q(s,a) \leftarrow Q(s,a) + \alpha\left(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right)$
9: $s := s'$
10: **until** s' is not the terminal state
11: **end for**

Step 1 Initialize Q-values:

- Initialize **Q(s,a) randomly or with zeros.**

Step 2 Start Episode

- **Choose starting state:** s

Step 3 Choose Action

Select action:

- **using current Q-values,**
- **plus exploration strategy.**

Step 4 Execute Action

- **Take action a**

Step 5 Observe Environment

Observe:

- **next state (s')**
- **reward (r)**

Step 6️⃣ Update Q-value

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_a Q(s',a) - Q(s,a)]$$

Step 7️⃣ Move to Next State

- Update: **s=s'**

Step 8️⃣ Repeat

Continue until:

- **terminal/goal state reached.**

📈 Why Q-Learning Works

Over time:

- **good actions receive high Q-values,**
- **bad** actions receive **low** Q-values.

Eventually: **optimal path emerges.**

- Q-Learning is: **Off-Policy RL**
  - it **learns the optimal policy, regardless of the actions currently taken.**
- Even random exploration can still lead to optimal learning.
- The algorithm tries to learn: $Q^*(s,a)$ which is the **optimal Q-value.**
- The optimal policy is: $\pi^*(s) = \arg\max_a Q^*(s,a)$
- The best action is: **the one with highest optimal Q-value.**

Q-Learning **converges to optimal policy** if:

1️⃣Infinite Visits

- **Every state-action pair** is **visited many times**
- **ensures proper learning.**

2️⃣Learning Rate Decays

- **Learning rate** should **gradually decrease over time.**

Conditions:

$$\sum_{t=0}^{\infty} \alpha_t = \infty \quad \& \quad \sum_{t=0}^{\infty} \alpha_t^2 < \infty$$

These mathematical conditions **guarantee convergence.**

In practice:

- policies often converge much earlier, before exact Q-values fully converge.

This makes Q-learning:

- practical,
- efficient,
- widely usable.

🌍 Exploration Strategy

- If the **agent always chooses: current best action**, it may miss: better unseen paths. So exploration is necessary.

Exploration	Try new actions
Exploitation	Use best known action

RL balances both

⭐ Epsilon-Greedy Strategy

Most common exploration method: **ε-Greedy Policy**

Probability	Action
ε	Random action
(1 - ε)	Best Q-value action

If: epsilon = 0.1

Then:

- ***10%*** → *random exploration*
- ***90%*** → *best known action*

🧠 Why ε-Greedy Works

It allows:

- discovering better states,
- avoiding local optimum,
- visiting more state-action pairs.

📘 Complete Intuition of Q-Learning

The agent repeatedly asks:

- "Was this action good?"
- "Could future rewards make it even better?"

Then it updates its knowledge accordingly.

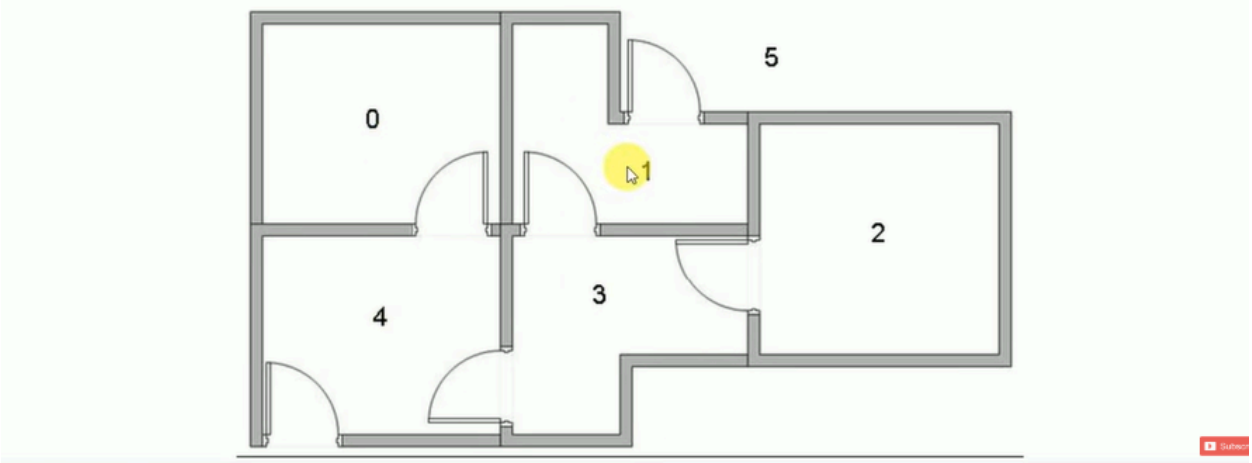
Over time:

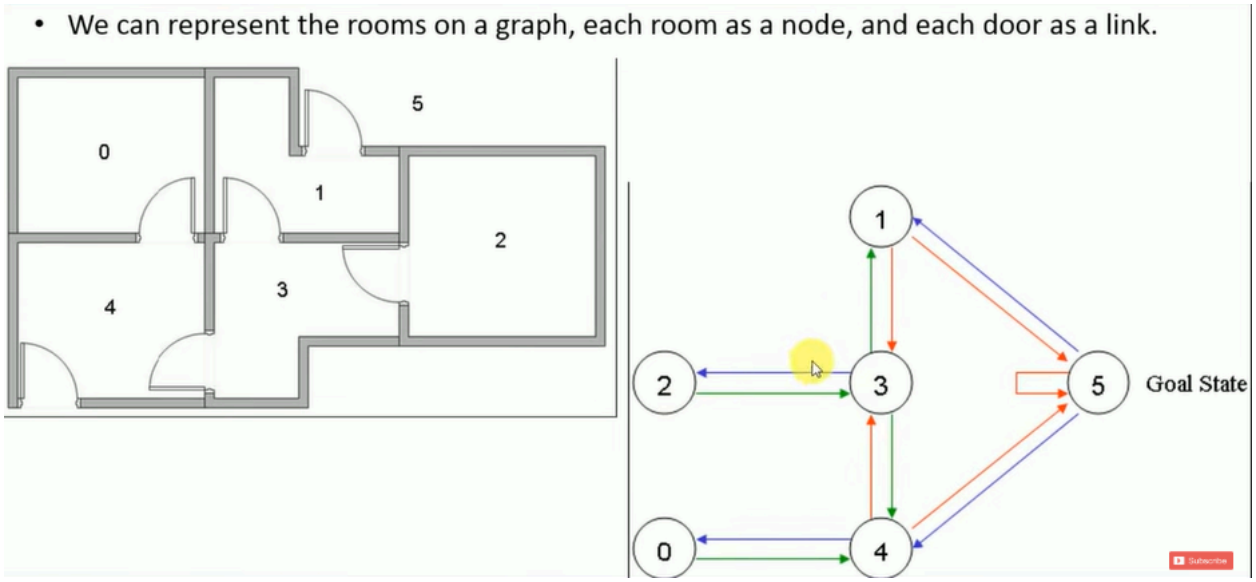
- optimal actions dominate,
- poor actions disappear.

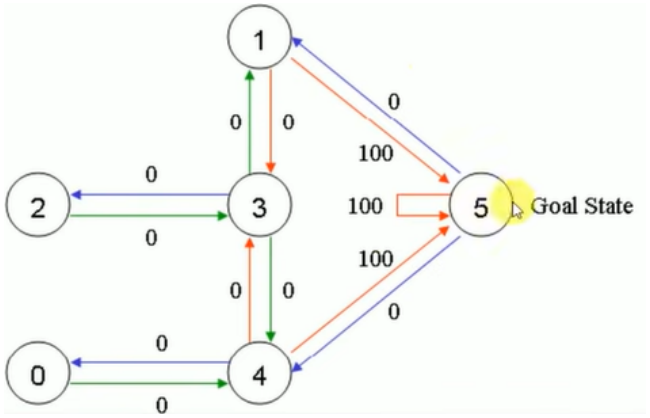
🤖 Q-Learning Example — Room Navigation Problem

https://www.youtube.com/watch?v=J3qX50yyiU0

- Suppose we have 5 rooms in a building connected by doors as shown in the figure below. We'll number each room 0 through 4. The outside of the building can be thought of as one big room (5). Notice that doors 1 and 4 lead into the building from room 5 (outside).







$$R = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Action} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array}$$

› Reward Matrix R:

This matrix tells us: **"What reward do we get for moving from one room to another?"**
› basically a map of: allowed moves, blocked moves, goal rewards.

- **R[1,5]=100** ⇒ Going from room 1 → room 5 gives reward 100.

-1	Invalid move
0	Valid move but no reward
100	Goal reached

Q learning algorithm

For each s, a initialize the table entry $\hat{Q}(s, a)$ to zero.
Observe the current state s
Do forever:

- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$

- Learning rate = 0.8 and the initial state as Room 1.
- Initialize matrix Q as a zero matrix:

› Q Matrix:

- "How useful is moving from one room to another?"
- Initially 0 ⇒ because the agent knows nothing.

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$Q(s, a) = R(s, a) + \gamma \max Q(s', a')$$

- Look at the second row (state 1) of matrix R.
- There are two possible actions for the current state 1: go to state 3, or go to state 5.
- By random selection, we select to go to 5 as our action.

› if we select 5 ⇒ we get **immediate reward of 100**

- Now let's imagine what would happen if our agent were in state 5 (next state).
- Look at the sixth row of the reward matrix R (i.e. state 5).
- It has 3 possible actions: go to state 1, 4 or 5.
- Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]
- Q(1, 5) = R(1, 5) + 0.8 * Max[Q(5, 1), Q(5, 4), Q(5, 5)] = 100 + 0.8 * 0 = 100

State \ Action	0	1	2	3	4	5
0	−1	−1	−1	−1	0	−1
1	−1	−1	−1	0	−1	100
2	−1	−1	−1	0	−1	−1
3	−1	0	0	−1	0	−1
4	0	−1	−1	0	−1	100
5	−1	0	−1	−1	0	100

$R =$

$Q =$

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

- The next state, 5, now becomes the current state.
- Because 5 is the goal state, we've finished one episode.
- Our agent's brain now contains an updated matrix Q as:

$Q =$

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	100
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

- For the next episode, we randomly choose the initial state – say 3 (can go to 1, 2 & 4)

State \ Action	0	1	2	3	4	5
0	−1	−1	−1	−1	0	−1
1	−1	−1	−1	0	−1	100
2	−1	−1	−1	0	−1	−1
3	−1	0	0	−1	0	−1
4	0	−1	−1	0	−1	100
5	−1	0	−1	−1	0	100

$R =$

- Now we imagine that we are in state 1 (next state).

- Now we imagine that we are in state 1 (next state).
- Look at the second row of reward matrix R (i.e. state 1).
- It has 2 possible actions: go to state 3 or state 5.
- Then, we compute the Q value:
- Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]
- Q(3, 1) = R(3, 1) + 0.8 * Max[Q(1, 3), Q(1, 5)] = 0 + 0.8 * Max(0, 100) = 80

$$R = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} \text{Action} & & & & & \\ 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array} \qquad Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

> repeat this baar baar, n we get our Q matrix

- If our agent learns more through further episodes, it will finally reach convergence values in matrix Q like:

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{array}$$

- Tracing the best sequences of states is as simple as following the links with the highest values at each state.